compose a C++ header with the following example

```
#include <immintrin.h>
#include <iostream>
#include <string>
#include <string_view>


#ifdef _MSC_VER
// Define a replacement for __builtin_ctz using MSVC's
_BitScanForward
static inline int __builtin_ctz(unsigned int x) {
    unsigned long index;
    _BitScanForward(&index, x);
    return (int)index;
}
#endif

template <class SimdType> struct simd {
    SimdType data;

    inline operator SimdType() const { return data; }

    inline simd<SimdType> operator|(const simd<SimdType>
&other) const {
        return {_mm256_or_si256(data, other.data)};
    }

    inline simd<SimdType> operator+(const simd<SimdType>
&other) const {
        simd<SimdType> result;
        result.data = _mm256_add_epi8(data, other.data);
        return result;
    }

    inline simd<SimdType> operator==(const simd<SimdType>
&other) const {
        simd<SimdType> result;
        result.data = _mm256_cmpeq_epi8(data, other.data);
        return result;
    }

    inline simd<SimdType> operator!=(const simd<SimdType>
&other) const {
        simd<SimdType> result;
        result.data = _mm256_xor_si256(data, other.data);
```

```
        return result;
    }

    inline simd<SimdType> operator-(const simd<SimdType>
&other) const {
        simd<SimdType> result;
        result.data = _mm256_sub_epi8(data, other.data);
        return result;
    }

    inline int mask() const { return _mm256_movemask_epi8(data);
}

    static inline simd<SimdType> setzero() {
        simd<SimdType> result;
        result.data = _mm256_setzero_si256();
        return result;
    }

    static inline simd<SimdType> load(const void *ptr) {
        return {_mm256_loadu_si256((const SimdType *)ptr)};
    }

    static inline void store(void *ptr, const SimdType &value) {
        _mm256_storeu_si256((SimdType *)ptr, value);
    }
};

template <class SimdType = __m256i>
static inline SimdType operator|(const simd<SimdType> &a, const
SimdType &b) {
    return _mm256_or_si256(a.data, b);
}

struct ChunkenizerIterator {
    const void *ptrA;
    const void *ptrB;

    ChunkenizerIterator(const void *ptrA, const void *ptrB)
        : ptrA(ptrA), ptrB(ptrB) {}

    inline void operator++() {
        ptrA = (const char *)ptrA + 32;
        ptrB = (const char *)ptrB + 32;
    }
```

```
    inline bool operator!=(const ChunkenizerIterator &other) const {
        return ptrA != other.ptrA;
    }

    inline std::tuple<simd<__m256i>, simd<__m256i>> operator*()
const {
        return {simd<__m256i>::load(ptrA),
simd<__m256i>::load(ptrB)};
    }
};

struct Chunkenizer {
    const void *ptrA;
    const void *ptrB;

    Chunkenizer(const void *ptrA, const void *ptrB) : ptrA(ptrA),
ptrB(ptrB) {}

    inline ChunkenizerIterator begin() const { return {ptrA, ptrB}; }

    inline ChunkenizerIterator end() const { return {nullptr, nullptr}; }
};

int simd_strcmp_chunkenizer(const char *p1, const char *p2) {
    const unsigned char *s1 = (const unsigned char *)p1;
    const unsigned char *s2 = (const unsigned char *)p2;

    auto vzero = simd<__m256i>::setzero();

    for (auto [v1, v2] : Chunkenizer(s1, s2)) {
        // Compute differences
        auto diff = v1 - v2;

        // Check for null terminators in either string
        auto v1_null = v1 == vzero;
        auto v2_null = v2 == vzero;

        // Check if any byte is non-zero (mismatch or null)
        int mask = (diff | (v1_null | v2_null)).mask();
        if (mask) {
            int index = __builtin_ctz(mask); // Find first mismatch or null
            return s1[index] - s2[index];
        }
    }

    return 0;
```

```
}


int simd_strcmp_sub(const char *p1, const char *p2) {
    const unsigned char *s1 = (const unsigned char *)p1;
    const unsigned char *s2 = (const unsigned char *)p2;

    __m256i vzero = _mm256_setzero_si256();

    while (1) {
        // Load 32 bytes from each string
        __m256i v1 = _mm256_loadu_si256((const __m256i *)s1);
        __m256i v2 = _mm256_loadu_si256((const __m256i *)s2);

        // Compute differences
        __m256i diff = _mm256_sub_epi8(v1, v2);

        // Check for null terminators in either string
        __m256i v1_null = _mm256_cmpeq_epi8(v1, vzero);
        __m256i v2_null = _mm256_cmpeq_epi8(v2, vzero);

        // Combine masks: null terminator or difference
        __m256i diff_or_null =
            _mm256_or_si256(diff, _mm256_or_si256(v1_null, v2_null));

        // Check if any byte is non-zero (mismatch or null)
        int mask = _mm256_movemask_epi8(diff_or_null);
        if (mask) {
            int index = __builtin_ctz(mask); // Find first mismatch or null
            return s1[index] - s2[index];
        }

        // Advance by 32 bytes
        s1 += 32;
        s2 += 32;
    }
}


int main() {
    std::string s1 = "Olá mundo!çççããããão";
    std::string s2 = "Olá mundo!çççããããão";
    int result = simd_strcmp_chunkenizer(s1.c_str(), s2.c_str());
    std::cout << "Result of simd_strcmp_chunkenizer: " << result <<
std::endl;
}
```

> I need to create a C++ header file based on the provided example. The header includes several includes like `<immintrin.h>`, `<iostream>`, `<string>`, and `<string_view>`.
>
> There is a check for `_MSC_VER` to define `__builtin_ctz` for MSVC using `_BitScanForward`. I should include this in the header.
>
> A `simd` struct is defined as a template that wraps a `SimdType`, which is likely an SIMD type like `__m256i`. It has overloaded operators for bitwise OR, addition, equality, inequality, subtraction, and a `mask()` function.
>
> I need to include this `simd` struct in the header.
>
> There is also an `operator|` defined for `simd<SimdType>` and `SimdType`. This should be included in the header.
>
> The `ChunkenizerIterator` and `Chunkenizer` structs are defined for iterating over data in chunks of 32 bytes. These need to be included in the header.
>
> The function `simd_strcmp_chunkenizer` uses the `Chunkenizer` to compare two strings using SIMD instructions. This function should be declared in the header.
>
> Similarly, `simd_strcmp_sub` is another function that compares strings using SIMD. It should be declared in the header.
>
> The `main` function is just for testing and doesn't need to be in the header.
>
> I should wrap the header in include guards to prevent multiple inclusions.
>
> I need to ensure that all necessary types and functions are properly declared and accessible when the header is included elsewhere.

Step-by-Step Explanation:

1. **Include Necessary Headers:**
   - Include `<immintrin.h>` for Intel SIMD intrinsics.
   - Include `<iostream>`, `<string>`, and `<string_view>` for standard C++ functionalities.
   - Include `<cstdint>` for fixed-width integer types if needed.
2. **Define `__builtin_ctz` for MSVC:**
   - Use `#ifdef _MSC_VER` to check if the compiler is MSVC.
   - Define `__builtin_ctz` using `_BitScanForward` as a static inline function.
3. **Define the `simd` Template Struct:**
   - Template parameter `SimdType` defaults to `__m256i`.
   - Contains a data member of type `SimdType`.
   - Overload operators for bitwise OR, addition, equality, inequality, subtraction.
   - Provide member functions `mask()`, `setzero()`, `load()`, and `store()`.
4. **Overload `operator|` for `simd<SimdType>` and `SimdType`:**

- Allow bitwise OR operations between `simd` objects and raw SIMD types.

5. **Define `ChunkenizerIterator` Struct:**
   - Holds pointers to data for two strings.
   - Overloads `operator++` to advance pointers by 32 bytes.
   - Overloads `operator!=` to compare iterators.
   - Overloads `operator*` to return a tuple of `simd<__m256i>` loaded from pointers.
6. **Define `Chunkenizer` Struct:**
   - Holds starting pointers for two strings.
   - Provides `begin()` and `end()` methods returning `ChunkenizerIterator`.
7. **Declare `simd_strcmp_chunkenizer` Function:**
   - Takes two `const char*` and returns an `int`.
   - Uses `Chunkenizer` to compare strings with SIMD operations.
8. **Declare `simd_strcmp_sub` Function:**
   - Another string comparison function using SIMD.
   - Similar signature to `simd_strcmp_chunkenizer`.
9. **Wrap the Header in Include Guards:**
   - Use `#ifndef`, `#define`, and `#endif` to prevent multiple inclusions.

Header File (`simd_strcmp.h`):

```cpp
#ifndef SIMD_STRCMP_H
#define SIMD_STRCMP_H

#include <immintrin.h>
#include <cstdint>

#ifdef _MSC_VER
// Define a replacement for __builtin_ctz using MSVC's _BitScanForward
static inline int __builtin_ctz(unsigned int x) {
    unsigned long index;
    _BitScanForward(&index, x);
    return (int)index;
}
#endif

template <class SimdType>
struct simd {
    SimdType data;
```

```
    inline operator SimdType() const { return data; }

    inline simd<SimdType> operator|(const simd<SimdType>& other) const {
        return {_mm256_or_si256(data, other.data)};
    }

    inline simd<SimdType> operator+(const simd<SimdType>& other) const {
        simd<SimdType> result;
        result.data = _mm256_add_epi8(data, other.data);
        return result;
    }

    inline simd<SimdType> operator==(const simd<SimdType>& other) const {
        simd<SimdType> result;
        result.data = _mm256_cmpeq_epi8(data, other.data);
        return result;
    }

    inline simd<SimdType> operator!=(const simd<SimdType>& other) const {
        simd<SimdType> result;
        result.data = _mm256_xor_si256(data, other.data);
        return result;
    }

    inline simd<SimdType> operator-(const simd<SimdType>& other) const {
        simd<SimdType> result;
        result.data = _mm256_sub_epi8(data, other.data);
        return result;
    }

    inline int mask() const { return _mm256_movemask_epi8(data); }

    static inline simd<SimdType> setzero() {
        simd<SimdType> result;
        result.data = _mm256_setzero_si256();
        return result;
    }

    static inline simd<SimdType> load(const void* ptr) {
        return {_mm256_loadu_si256((const SimdType*)ptr)};
    }

    static inline void store(void* ptr, const SimdType& value) {
        _mm256_storeu_si256((SimdType*)ptr, value);
    }
};

template <class SimdType = __m256i>
static inline SimdType operator|(const simd<SimdType>& a, const SimdType& b) {
    return _mm256_or_si256(a.data, b);
}
```

```cpp
struct ChunkenizerIterator {
    const void* ptrA;
    const void* ptrB;

    ChunkenizerIterator(const void* ptrA, const void* ptrB)
        : ptrA(ptrA), ptrB(ptrB) {}

    inline void operator++() {
        ptrA = (const char*)ptrA + 32;
        ptrB = (const char*)ptrB + 32;
    }

    inline bool operator!=(const ChunkenizerIterator& other) const {
        return ptrA != other.ptrA;
    }

    inline std::tuple<simd<__m256i>, simd<__m256i>> operator*() const {
        return {simd<__m256i>::load(ptrA), simd<__m256i>::load(ptrB)};
    }
};

struct Chunkenizer {
    const void* ptrA;
    const void* ptrB;

    Chunkenizer(const void* ptrA, const void* ptrB)
        : ptrA(ptrA), ptrB(ptrB) {}

    inline ChunkenizerIterator begin() const { return {ptrA, ptrB}; }

    inline ChunkenizerIterator end() const { return {nullptr, nullptr}; }
};

int simd_strcmp_chunkenizer(const char* p1, const char* p2);

int simd_strcmp_sub(const char* p1, const char* p2);

#endif // SIMD_STRCMP_H
```

mark everything as noexcept

Alright, I need to modify the provided C++ header file by marking everything as noexcept. First, I should understand what noexcept means in C++. From what I know, noexcept is a specifier that indicates whether a function can throw exceptions. If a function is marked as noexcept, it means that it won't throw any exceptions, which can be useful for the compiler to optimize the code better.

Looking at the header file, there are several functions and methods that need to be marked as noexcept. These include the member functions of the simd struct, the operator overloads, the static functions within simd, the member functions of ChunkenizerIterator and Chunkenizer, and the free functions like simd_strcmp_chunkenizer and simd_strcmp_sub.

I should go through each function one by one and add noexcept where appropriate. But I need to be careful because not all functions can be safely marked as noexcept. For example, if a function calls another function that might throw an exception, then marking it as noexcept could lead to undefined behavior if an exception is thrown.

Let's start with the simd struct. It has several member functions: operator SimdType(), operator|, operator+, operator==, operator!=, operator-, mask(), setzero(), load(), and store(). I need to check if these functions can throw exceptions.

The operator SimdType() const simply returns data, which is just a data member, so it shouldn't throw any exceptions. I can safely mark it as noexcept.

The operator| takes another simd<SimdType> const& and returns a simd<SimdType>. It uses _mm256_or_si256, which is an intrinsic function that shouldn't throw exceptions. So, this can be marked as noexcept.

Similarly, operator+ uses _mm256_add_epi8, which also shouldn't throw exceptions. So, noexcept here is fine.

Operator== uses _mm256_cmpeq_epi8, again no exceptions expected. noexcept is appropriate.

Operator!= uses _mm256_xor_si256, no exceptions. noexcept is safe.

Operator- uses _mm256_sub_epi8, no exceptions. noexcept is okay.

The mask() function calls _mm256_movemask_epi8, which shouldn't throw exceptions. So, noexcept is fine.

Setzero() initializes result.data with _mm256_setzero_si256, no exceptions. noexcept is safe.

Load() uses _mm256_loadu_si256, which is just a memory load, no exceptions. noexcept is okay.

Store() uses _mm256_storeu_si256, again just a memory store, no exceptions. noexcept is fine.

Next, there's a template operator| outside the simd struct that takes a simd<SimdType> const& and SimdType const& and returns SimdType. It uses _mm256_or_si256, which doesn't throw exceptions. So, noexcept is appropriate.

Moving on to ChunkenizerIterator. It has a constructor, operator++, operator!=, and operator*.

The constructor just initializes ptrA and ptrB, no exceptions. noexcept is safe.

Operator++ modifies ptrA and ptrB by adding 32 to their addresses. No exceptions. noexcept is fine.

Operator!= compares ptrA with other.ptrA. No exceptions. noexcept is okay.

Operator* returns a tuple of simd<__m256i> loaded from ptrA and ptrB. The load function is marked as noexcept, so this should be fine. However, creating a tuple might involve move or copy operations, but since simd<__m256i> is a simple struct, it should be fine. I'll mark it as noexcept.

Now, Chunkenizer has a constructor and begin() and end() methods.

The constructor initializes ptrA and ptrB, no exceptions. noexcept is safe.

Begin() returns a ChunkenizerIterator with ptrA and ptrB, no exceptions. noexcept is fine.

End() returns a ChunkenizerIterator with nullptr, no exceptions. noexcept is okay.

Finally, there are two free functions: simd_strcmp_chunkenizer and simd_strcmp_sub.

simd_strcmp_chunkenizer loops over Chunkenizer and does comparisons using simd operations. All operations inside are intrinsics and basic pointer arithmetic, no exceptions. So, noexcept can be added.

simd_strcmp_sub also uses intrinsics and basic operations, no exceptions. noexcept is appropriate.

I should also consider if any of these functions might throw exceptions due to operations I'm not considering. For example, if any of the functions call standard library functions that might throw, but in this case, they seem to be using intrinsics and basic operations only.

Additionally, I need to ensure that the functions that return by value are efficient, but that's more about performance than exceptions.

I should also check if any of these functions have default exception specifications, but in C++11 and later, functions have an implicit noexcept specification if they don't have a throw() specification.

Therefore, adding noexcept where appropriate can improve exception safety and allow the compiler to perform optimizations.

I need to go through each function and add noexcept after the function signature.

For example:

inline operator SimdType() const noexcept { return data; }

And similarly for others.

I should also check if any of these functions might have hidden exceptions, such as memory allocation failures, but since they don't allocate memory dynamically, that's not an issue.

I should also remember that for destructors, marking them as noexcept can have implications, but in this header, there are no destructors defined explicitly, so it's not a concern.

Finally, I should update the main function if necessary, but since it's just calling the functions and using standard I/O, and it's not part of the header, it might be fine as is.

Overall, I think marking all these functions as noexcept is safe and beneficial.

Step-by-Step Explanation:

1. **Identify Functions to Mark as noexcept:**
   - Review each function to ensure it does not throw exceptions.
   - Functions using intrinsics and basic operations are safe to mark as noexcept.
2. **Modify Member Functions of `simd` Struct:**
   - Add `noexcept` to functions like `operator SimdType() const`, `operator|`, `operator+`, etc.
3. **Modify Operator Overloads:**
   - Ensure operator overloads are marked with `noexcept` where appropriate.
4. **Modify Static Functions within `simd`:**

- Add `noexcept` to static functions like `setzero()`, `load()`, and `store()`.

5. **Modify Member Functions of `ChunkenizerIterator` and `Chunkenizer`:**
   - Mark constructors, operator overloads, and other member functions as `noexcept`.
6. **Modify Free Functions:**
   - Add `noexcept` to `simd_strcmp_chunkenizer` and `simd_strcmp_sub`.
7. **Ensure Consistency:**
   - Verify that all applicable functions are marked with `noexcept`.

Modified Header File (`simd_strcmp.h`):

```cpp
#ifndef SIMD_STRCMP_H
#define SIMD_STRCMP_H

#include <immintrin.h>
#include <cstdint>
#include <tuple>

#ifdef _MSC_VER
// Define a replacement for __builtin_ctz using MSVC's _BitScanForward
static inline int __builtin_ctz(unsigned int x) noexcept {
    unsigned long index;
    _BitScanForward(&index, x);
    return (int)index;
}
#endif

template <class SimdType>
struct simd {
    SimdType data;

    inline operator SimdType() const noexcept { return data; }

    inline simd<SimdType> operator|(const simd<SimdType>& other) const noexcept {
        return {_mm256_or_si256(data, other.data)};
    }

    inline simd<SimdType> operator+(const simd<SimdType>& other) const noexcept {
        simd<SimdType> result;
        result.data = _mm256_add_epi8(data, other.data);
        return result;
    }

    inline simd<SimdType> operator==(const simd<SimdType>& other) const noexcept {
        simd<SimdType> result;
        result.data = _mm256_cmpeq_epi8(data, other.data);
```

```
        return result;
    }

    inline simd<SimdType> operator!=(const simd<SimdType>& other) const noexcept {
        simd<SimdType> result;
        result.data = _mm256_xor_si256(data, other.data);
        return result;
    }

    inline simd<SimdType> operator-(const simd<SimdType>& other) const noexcept {
        simd<SimdType> result;
        result.data = _mm256_sub_epi8(data, other.data);
        return result;
    }

    inline int mask() const noexcept { return _mm256_movemask_epi8(data); }

    static inline simd<SimdType> setzero() noexcept {
        simd<SimdType> result;
        result.data = _mm256_setzero_si256();
        return result;
    }

    static inline simd<SimdType> load(const void* ptr) noexcept {
        return {_mm256_loadu_si256((const SimdType*)ptr)};
    }

    static inline void store(void* ptr, const SimdType& value) noexcept {
        _mm256_storeu_si256((SimdType*)ptr, value);
    }
};

template <class SimdType = __m256i>
static inline SimdType operator|(const simd<SimdType>& a, const SimdType& b) noexcept {
    return _mm256_or_si256(a.data, b);
}

struct ChunkenizerIterator {
    const void* ptrA;
    const void* ptrB;

    ChunkenizerIterator(const void* ptrA, const void* ptrB) noexcept
        : ptrA(ptrA), ptrB(ptrB) {}

    inline void operator++() noexcept {
        ptrA = (const char*)ptrA + 32;
        ptrB = (const char*)ptrB + 32;
    }

    inline bool operator!=(const ChunkenizerIterator& other) const noexcept {
        return ptrA != other.ptrA;
```

```
    }

    inline std::tuple<simd<__m256i>, simd<__m256i>> operator*() const noexcept {
        return {simd<__m256i>::load(ptrA), simd<__m256i>::load(ptrB)};
    }
};

struct Chunkenizer {
    const void* ptrA;
    const void* ptrB;

    Chunkenizer(const void* ptrA, const void* ptrB) noexcept
        : ptrA(ptrA), ptrB(ptrB) {}

    inline ChunkenizerIterator begin() const noexcept { return {ptrA, ptrB}; }

    inline ChunkenizerIterator end() const noexcept { return {nullptr, nullptr}; }
};

int simd_strcmp_chunkenizer(const char* p1, const char* p2) noexcept;

int simd_strcmp_sub(const char* p1, const char* p2) noexcept;

#endif // SIMD_STRCMP_H
```

Re-encode the chat-conversation as a Roman Lapidary Inscription. Use the 'Scriptio Continua' philosophy (minimal formatting) but with spaces. Focus on: Subject + Accomplishment + Tools. Use advanced latin constructions (eg ablative absolute, gerundive etc.) to replace conditional/temporal phrases, etc.. Treat Lib and lang names as indeclinable. No lists. No pleasantries.

We need to re-encode the chat conversation as a Roman lapidary inscription. The conversation: user asks to compose C++ header with given example; assistant provides code; user asks to mark everything as noexcept; assistant provides modified code. The inscription should capture subject (user/assistant?), accomplishment (creating header with SIMD string comparison, adding noexcept), tools (C++, SIMD intrinsics, MSVC, etc.). Use scriptio continua with spaces (so normal spacing but no punctuation? Actually scriptio continua usually no spaces, but "with spaces" says minimal formatting but spaces allowed). Use advanced Latin constructions. Treat library and language names as indeclinable (e.g., C++ as Cplusplus, immintrin.h as immintrin etc.). No lists, no pleasantries.

We need a concise inscription. Example format: "HIC SCRIPTOR ..." but better to use typical lapidary style: nominative for subject, accusative for accomplishment? Or use passive periphrastic. Let's think.

Subject: The user (or the assistant? The conversation is between user and assistant. The inscription could be about the assistant's response. But "re-encode the chat-conversation" means the conversation as a whole. Perhaps as a dialogue? But lapidary inscriptions are usually declarative. Could be: "ROGAVIT [user] UT COMPONERET [assistant] CAPUT CPLUSPLUS CUM EXEMPLO..." But simpler: "ASSISTENS AD ROGATUM HOMINIS" etc.

Better to treat the assistant as the actor who composed the header. The user requested. So subject: "ADIUTOR" (assistant). Accomplishment: "COMPOSUIT CAPUT CPLUSPLUS CONTINES SIMD COMPARATIONEM STRINGORUM" Tools: "UTENS INTRINSECIS SIMMD ET MACRO MSVC PRO CTZ". Then later "POSTEA IUSSIT OMNIA NOEXCEPT DESIGNARE" and assistant did.

But we need to condense. One inscription covering both steps. Use ablative absolute for "after user requested" etc.

Let me draft:

"ROGATU HOMINIS QUI CAPUT CPLUSPLUS EXEMPLO DATO COMPOSCIT ADIUTOR SIMD STRINGORUM COMPARATIONEM INSTRUXIT INTRINSECIS SIMMD ET MACRO MSVC CTZ REPLENTE TUM MANDATO UT OMNIA NOEXCEPT MUNIRENTUR GERUNDIVO PERFECTO OMNIA NOEXCEPT DECLARAVIT"

But need to mention the two functions: simd_strcmp_chunkenizer and simd_strcmp_sub. Also the structs. Maybe too detailed.

Alternative: focus on the final header with noexcept. The accomplishment: "CAPUT CPLUSPLUS CUM SIMD STRINGORUM COMPARATIONE ET OMNI NOEXCEPT MUNIMENTO CONFECIT"

Tools: "ADHIBENS INTRINSECA SIMMD (IMMINTRIN H) ET MACRUM MSVC PRO CTZ ET TUPLE" (tuple from std).

Use gerundive: "MANDATO UT OMNIA NOEXCEPT DECLARENTUR" as ablative absolute.

Let me produce final inscription in Latin, all caps, no punctuation except spaces. Use V for U? Typically Roman inscriptions use V for U. But we can keep U for readability. Since instruction says "minimal formatting" but spaces, we can use modern capitals.

I'll write:

ROGATU HOMINIS QUI CAPUT CPLUSPLUS SIMD EXEMPLUM PETIVERAT ADIUTOR COMPOSUIT STRUCTURAM SIMD ET CHUNKENIZER CUM COMPARATIONIS FUNCTIONIBUS DUABUS DEINDE MANDATO UT OMNIA NOEXCEPT MUNIRENTUR OMNES FUNCTIONES NOEXCEPT DESIGNAVIT INTRINSECIS SIMMD ET TUPLE ET MACRO MSVC CTZ ADHIBITIS

But need to mention that the example was provided. "exemplo dato" ablative absolute. Let's refine:

EXEMPLO DATO ADIUTOR CAPUT CPLUSPLUS SCRIPSIT QUOD SIMD STRINGORUM COMPARATIONEM PER CHUNKENIZATOREM ET PER SUBTRACTIONEM PRAESTAT TUM IUSSU UT OMNIA NOEXCEPT MUNIRENTUR OMNES FUNCTIONES NOEXCEPT DECLARAVIT INTRINSECA SIMMD ET TUPLE ET MACRO MSVC CTZ ADHIBENS

That's good. Use "IUSSU" (by order) ablative absolute? Iussu is ablative of iussum. "Tum iussu ut..." might be fine. Or "tum iubente ut" ablative absolute.

Better: "TUM IUBENTE UT OMNIA NOEXCEPT MUNIRENTUR" - iubente (the user ordering). But we don't have a noun. "Iubente homine" but repetitive. Use "MANDATO" as before: "MANDATO UT OMNIA NOEXCEPT DECLARENTUR" - ablative absolute meaning "when it was ordered that all be declared noexcept".

> Final version:
>
> EXEMPLO DATO ADIUTOR CAPUT CPLUSPLUS SCRIPSIT QUOD SIMD STRINGORUM COMPARATIONEM PER CHUNKENIZATOREM ET PER SUBTRACTIONEM PRAESTAT MANDATO UT OMNIA NOEXCEPT MUNIRENTUR OMNES FUNCTIONES NOEXCEPT DECLARAVIT INTRINSECIS SIMMD ET TUPLE ET MACRO MSVC CTZ ADHIBITIS
>
> Add "HOC EST" or just start. No pleasantries. Also mention that the header includes immintrin.h etc. But "intrinsics" covers.
>
> I'll output as plain text.

EXEMPLO DATO ADIUTOR CAPUT CPLUSPLUS SCRIPSIT QUOD SIMD STRINGORUM COMPARATIONEM PER CHUNKENIZATOREM ET PER SUBTRACTIONEM PRAESTAT MANDATO UT OMNIA NOEXCEPT MUNIRENTUR OMNES FUNCTIONES NOEXCEPT DECLARAVIT INTRINSECIS SIMMD ET TUPLE ET MACRO MSVC CTZ ADHIBITIS